RDRS — THEORETICAL MAXIMUM

125-Feature Master Specification | V1–V5 | 5-Day Build Plan

Scope: a unified ransomware-focused security platform covering endpoint telemetry, behavioral detection, SOC operations, investigation, threat hunting, XDR-style correlation, controlled response, SOAR, AI-assisted analysis, recovery/rollback, storage visibility, resilience and deployment. Internmo requirements are the baseline, not the product boundary.

NON-NEGOTIABLE BUILD RULE

V1 = Day 1, V2 = Day 2, V3 = Day 3, V4 = Day 4, V5 = Day 5. These are development milestones, NOT Basic → Advanced → Enterprise quality levels.

STRICT ORDER: Feature N must reach 100/100 before Feature N+1 starts. 80%, 95%, 99% or 'good enough' never permits a jump. For each feature: implement → test → edge cases → integrate → validate → document → mark COMPLETE_100 → LOCK.

A day boundary never permits skipping an incomplete feature. If a feature is unfinished at the planned boundary, finish it first; quality is not reduced to preserve the calendar.

V1 — DAY 1 — Core + Endpoint Telemetry

F01 — Core Runtime

WHAT / HOW: Permanent modular runtime, lifecycle manager, startup/shutdown and failure recovery.

OUTPUT: stable running RDRS core.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F02 — Event Bus

WHAT / HOW: Centralized normalized event transport between sensors and consumers.

OUTPUT: unique, routable security events.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F03 — Worker/Queue Engine

WHAT / HOW: Background workers, queues and backpressure for burst processing.

OUTPUT: telemetry bursts do not freeze the application.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F04 — Configuration Engine

WHAT / HOW: Validated configuration for monitoring, detection, response and AI behavior.

OUTPUT: validated runtime configuration.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F05 — Policy Engine

WHAT / HOW: Enforceable monitoring, detection and response policies.

OUTPUT: decisions traceable to policy.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F06 — Structured Logging

WHAT / HOW: Machine-readable system, security and error logs.

OUTPUT: searchable structured records.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F07 — Audit Engine

WHAT / HOW: Record important system and analyst actions.

OUTPUT: who/what/when/result trail.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F08 — Platform Health

WHAT / HOW: Health state for sensors, DB, workers, queues and API.

OUTPUT: live component health.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F09 — File Create Detection

WHAT / HOW: Detect file creation.

OUTPUT: normalized CREATE event.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F10 — File Modify Detection

WHAT / HOW: Detect file modification with context.

OUTPUT: normalized MODIFY event.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F11 — File Delete Detection

WHAT / HOW: Detect file deletion.

OUTPUT: normalized DELETE event.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F12 — File Rename/Move Detection

WHAT / HOW: Track old and new paths.

OUTPUT: rename/move relationship.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F13 — Recursive Monitoring

WHAT / HOW: Monitor nested directories.

OUTPUT: complete configured tree visibility.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F14 — Monitoring Exclusions

WHAT / HOW: Exclude approved paths/patterns.

OUTPUT: controlled monitoring noise.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F15 — File Metadata

WHAT / HOW: Collect size, extension, timestamps, type and path.

OUTPUT: enriched file event.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F16 — SHA-256 Hashing

WHAT / HOW: Generate deterministic file identity.

OUTPUT: hash for integrity/IOC use.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F17 — File-Type Identification

WHAT / HOW: Identify file type/context beyond extension.

OUTPUT: extension/type mismatch context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F18 — Shannon Entropy

WHAT / HOW: Calculate 0–8 entropy using 64-KB sampling.

OUTPUT: entropy value.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F19 — Entropy History

WHAT / HOW: Store and compare entropy over time.

OUTPUT: entropy delta/history.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F20 — Process Creation

WHAT / HOW: Capture process creation metadata.

OUTPUT: PID, executable, parent context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F21 — Process Tree

WHAT / HOW: Maintain parent-child process lineage.

OUTPUT: execution chain.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F22 — Process Resource Telemetry

WHAT / HOW: Collect CPU, memory, disk I/O and runtime.

OUTPUT: process behavior snapshot.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F23 — Command-Line Telemetry

WHAT / HOW: Capture command-line execution context where supported.

OUTPUT: process invocation context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F24 — User/Session Telemetry

WHAT / HOW: Capture user, session and privilege context.

OUTPUT: user → session → process attribution.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F25 — Network Connection Telemetry

WHAT / HOW: Capture destination, port, protocol, DNS and process context.

OUTPUT: process → network relationship.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

V2 — DAY 2 — Ransomware Detection + Response

F26 — Event Normalization

WHAT / HOW: Convert raw sensor events to a common schema.

OUTPUT: consistent RDRS events.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F27 — Event Deduplication

WHAT / HOW: Identify repeated sensor events.

OUTPUT: clean telemetry.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F28 — Sliding-Window Analysis

WHAT / HOW: Analyze recent events over configurable windows.

OUTPUT: rates and trends.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F29 — Modification-Burst Detection

WHAT / HOW: Detect abnormal file modification velocity.

OUTPUT: modification anomaly.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F30 — Mass-Rename Detection

WHAT / HOW: Detect large-scale rename behavior.

OUTPUT: rename anomaly.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F31 — Extension-Transformation Detection

WHAT / HOW: Detect suspicious extension changes.

OUTPUT: extension attack signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F32 — Encryption-Like Behavior Detection

WHAT / HOW: Correlate rapid content transformation and activity.

OUTPUT: encryption confidence.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F33 — File-Access Anomaly Detection

WHAT / HOW: Detect abnormal read/write/access patterns.

OUTPUT: file-behavior anomaly.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F34 — Process/File Correlation

WHAT / HOW: Associate processes with files they modify.

OUTPUT: responsible process attribution.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F35 — User/File Correlation

WHAT / HOW: Associate users with file activity.

OUTPUT: user attribution.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F36 — Process/Network Correlation

WHAT / HOW: Associate suspicious processes with network activity.

OUTPUT: process + network evidence.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F37 — Host Behavioral Baseline

WHAT / HOW: Establish normal host activity.

OUTPUT: host baseline.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F38 — User Behavioral Baseline

WHAT / HOW: Establish normal user activity.

OUTPUT: user baseline.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F39 — Process Behavioral Baseline

WHAT / HOW: Establish normal process behavior.

OUTPUT: process baseline.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F40 — Anomaly Detection

WHAT / HOW: Calculate deviations from baselines.

OUTPUT: anomaly score and reason.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F41 — Detection Rule Engine

WHAT / HOW: Versioned rules, thresholds, severity and metadata.

OUTPUT: triggered rule + evidence.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F42 — Rule Testing

WHAT / HOW: Run detection rules against controlled test data.

OUTPUT: PASS/FAIL + detection result.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F43 — Rule Tuning

WHAT / HOW: Manage thresholds, exceptions and suppression.

OUTPUT: controlled false-positive behavior.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F44 — Signature Detection

WHAT / HOW: Match known malicious patterns.

OUTPUT: signature match.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F45 — IOC Detection

WHAT / HOW: Match hashes, IPs, domains, URLs and process indicators.

OUTPUT: IOC hit + source/confidence.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F46 — Pre-Encryption Detection

WHAT / HOW: Detect ransomware preparation before mass encryption.

OUTPUT: pre-impact alert.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F47 — Recovery-Tampering Detection

WHAT / HOW: Detect suspicious recovery/shadow-copy targeting.

OUTPUT: recovery attack signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F48 — Security-Control Tampering Detection

WHAT / HOW: Detect security-control/service/configuration manipulation.

OUTPUT: defense-evasion signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F49 — Canary File Detection

WHAT / HOW: Monitor decoy files as high-confidence tripwires.

OUTPUT: canary trigger.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F50 — Canary Directory Detection

WHAT / HOW: Monitor decoy directory activity.

OUTPUT: directory deception trigger.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

V3 — DAY 3 — SOC + Hunting + Investigation

F51 — Threat Scoring

WHAT / HOW: Combine detection signals into explainable 0–100 risk.

OUTPUT: risk score + contributing signals.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F52 — Confidence Engine

WHAT / HOW: Calculate confidence from evidence and signal quality.

OUTPUT: confidence value + basis.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F53 — Alert Generation

WHAT / HOW: Convert detections into analyst-ready alerts.

OUTPUT: structured alert.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F54 — Alert Deduplication

WHAT / HOW: Collapse repeated alerts.

OUTPUT: cleaner SOC queue.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F55 — Alert Suppression

WHAT / HOW: Suppress approved benign patterns.

OUTPUT: controlled alert noise.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F56 — Alert Prioritization

WHAT / HOW: Rank alerts by risk, confidence and scope.

OUTPUT: prioritized queue.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F57 — Alert Enrichment

WHAT / HOW: Attach process, file, user and network context.

OUTPUT: context-rich alert.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F58 — Incident Creation

WHAT / HOW: Group related alerts into an incident.

OUTPUT: incident ID + scope.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F59 — Incident Lifecycle

WHAT / HOW: Track open, investigate, contain, recover and close states.

OUTPUT: complete incident state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F60 — Case Management

WHAT / HOW: Notes, tasks, ownership, status and history.

OUTPUT: analyst case workspace.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F61 — Evidence Acquisition

WHAT / HOW: Collect incident-relevant evidence.

OUTPUT: incident-linked evidence package.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F62 — Evidence Hashing

WHAT / HOW: Hash collected evidence.

OUTPUT: integrity hash.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F63 — Evidence Provenance

WHAT / HOW: Track source, acquisition and destination.

OUTPUT: provenance history.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F64 — Evidence Verification

WHAT / HOW: Verify evidence integrity.

OUTPUT: PASS/FAIL integrity state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F65 — Attack Timeline

WHAT / HOW: Chronologically reconstruct relevant events.

OUTPUT: attack chronology.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F66 — Process Investigation

WHAT / HOW: Investigate process tree, command line and related activity.

OUTPUT: execution path.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F67 — File Investigation

WHAT / HOW: Pivot file → hash → process → event → incident.

OUTPUT: file-centric investigation.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F68 — Network Investigation

WHAT / HOW: Pivot connection → process → host → incident.

OUTPUT: network context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F69 — User Investigation

WHAT / HOW: Pivot user → session → process → activity.

OUTPUT: identity context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F70 — Attack Graph

WHAT / HOW: Represent relationships among entities.

OUTPUT: interactive attack graph.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F71 — Threat Hunting Search

WHAT / HOW: Search historical telemetry.

OUTPUT: matching entities/events.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F72 — Advanced Query Engine

WHAT / HOW: Boolean logic, filters, time ranges and aggregation.

OUTPUT: analyst-defined hunt results.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F73 — Saved Hunts

WHAT / HOW: Store reusable hunting queries.

OUTPUT: repeatable hunts.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F74 — Hunt Pivoting

WHAT / HOW: Pivot from results to related entities/evidence.

OUTPUT: investigation path.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F75 — Cross-Host Hunting

WHAT / HOW: Search the same behavior/IOC across endpoints.

OUTPUT: campaign scope.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

V4 — DAY 4 — XDR + AI + Response + Recovery

F76 — Cross-User Hunting

WHAT / HOW: Search behavior across users.

OUTPUT: user/campaign scope.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F77 — Cross-Process Hunting

WHAT / HOW: Search process behavior across hosts.

OUTPUT: process campaign scope.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F78 — MITRE ATT&CK Mapping

WHAT / HOW: Map detections/incidents to tactics, techniques and sub-techniques.

OUTPUT: ATT&CK mapping.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F79 — Detection Coverage

WHAT / HOW: Measure technique coverage and gaps.

OUTPUT: coverage/gap report.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F80 — Threat Intelligence Enrichment

WHAT / HOW: Add reputation and threat context to indicators.

OUTPUT: enriched indicator.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F81 — Threat Intelligence Entity View

WHAT / HOW: Show indicator relationships and related incidents.

OUTPUT: unified entity view.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F82 — SMB/File-Share Investigation

WHAT / HOW: Correlate share activity with host/user/process.

OUTPUT: share attack scope.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F83 — Identity Attack-Path Analysis

WHAT / HOW: Analyze identity, privilege and suspicious access relationships.

OUTPUT: identity attack path.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F84 — SOC Dashboard

WHAT / HOW: Functional operational GUI for alerts, incidents and telemetry.

OUTPUT: live SOC console.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F85 — Live Security Visualization

WHAT / HOW: Visualize live events, alerts, processes, files, network and score.

OUTPUT: live security view.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F86 — Analyst CLI

WHAT / HOW: Operational CLI using the same core.

OUTPUT: command-driven SOC operations.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F87 — REST API

WHAT / HOW: Expose RDRS operational capabilities programmatically.

OUTPUT: structured API responses.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F88 — Real-Time Event API

WHAT / HOW: Stream live security events.

OUTPUT: real-time event feed.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F89 — Notification Engine

WHAT / HOW: Generate operational/security notifications.

OUTPUT: notification events.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F90 — Escalation Engine

WHAT / HOW: Escalate unresolved high-severity incidents.

OUTPUT: escalation state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F91 — SOAR Playbook Engine

WHAT / HOW: Execute multi-step response workflows.

OUTPUT: playbook execution/result.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F92 — Approval Workflow

WHAT / HOW: Require approval for sensitive actions.

OUTPUT: approved/rejected action record.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F93 — Response Verification

WHAT / HOW: Verify response success/failure.

OUTPUT: verified response state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F94 — Endpoint Containment

WHAT / HOW: Support controlled endpoint isolation architecture.

OUTPUT: containment state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F95 — Process Containment

WHAT / HOW: Perform controlled process response.

OUTPUT: process containment result.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F96 — Network Containment

WHAT / HOW: Perform controlled network blocking/isolation.

OUTPUT: network containment result.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F97 — Identity Containment

WHAT / HOW: Support controlled account/session containment.

OUTPUT: identity containment result.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F98 — Storage/NAS Monitoring

WHAT / HOW: Monitor storage/share telemetry architecture.

OUTPUT: storage security events.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F99 — Storage I/O Anomaly

WHAT / HOW: Detect abnormal storage read/write behavior.

OUTPUT: storage anomaly signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F100 — Cross-Domain XDR Correlation

WHAT / HOW: Correlate endpoint, network, identity, file, threat-intel and cloud/SaaS-ready signals.

OUTPUT: unified cross-domain incident.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

V5 — DAY 5 — AI + Recovery + Enterprise Hardening

F101 — Cloud/SaaS Integration Layer

WHAT / HOW: Ingest cloud/application/SaaS security events through connectors.

OUTPUT: cloud events in common model.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F102 — Email Security Integration

WHAT / HOW: Bring email/attachment indicators into correlation.

OUTPUT: email → identity → endpoint context.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F103 — AI Security Assistant

WHAT / HOW: RDRS-aware GUI chatbot grounded in available telemetry/evidence.

OUTPUT: evidence-grounded answer.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F104 — AI Alert Triage

WHAT / HOW: Summarize, group and prioritize alerts.

OUTPUT: analyst-ready triage.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F105 — AI Incident Investigation

WHAT / HOW: Analyze timeline, process and evidence context.

OUTPUT: investigation narrative.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F106 — AI Natural-Language Hunting

WHAT / HOW: Convert natural language to controlled security queries.

OUTPUT: query + results + observations.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F107 — AI Attack-Chain Analysis

WHAT / HOW: Turn correlated telemetry into an attack story.

OUTPUT: attack-chain explanation.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F108 — AI Report Generation

WHAT / HOW: Generate technical and executive reports from case data.

OUTPUT: report draft/export.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F109 — AI Response Recommendation

WHAT / HOW: Recommend containment/playbook actions with reasoning.

OUTPUT: recommended action + rationale.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F110 — AI Agent Safety Boundary

WHAT / HOW: Enforce permissions, policy and approval around AI actions.

OUTPUT: auditable safe AI workflow.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F111 — Recovery-Point Engine

WHAT / HOW: Identify/manage available clean recovery points.

OUTPUT: recovery options.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F112 — File Recovery

WHAT / HOW: Restore eligible files and verify integrity.

OUTPUT: restored/not-restorable status.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F113 — Ransomware Rollback

WHAT / HOW: Track changes and perform controlled rollback workflow.

OUTPUT: affected/recoverable/restored counts.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F114 — UEBA

WHAT / HOW: Detect anomalous user/entity behavior.

OUTPUT: entity risk signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F115 — Data-Exfiltration Context

WHAT / HOW: Detect suspicious large/unusual outbound movement context.

OUTPUT: possible exfiltration signal.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F116 — Security Posture

WHAT / HOW: Aggregate endpoint, detection, identity and policy posture.

OUTPUT: posture dashboard.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F117 — Detection Engineering Workbench

WHAT / HOW: Create → test → simulate → deploy → tune → version → retire detections.

OUTPUT: complete detection lifecycle.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F118 — False-Positive Management

WHAT / HOW: Track feedback, suppressions and exceptions.

OUTPUT: FP management state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F119 — False-Negative Validation

WHAT / HOW: Test known behaviors for detection gaps.

OUTPUT: missed-behavior/gap report.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F120 — Performance & Scalability

WHAT / HOW: Benchmark throughput, latency and resource usage.

OUTPUT: performance report.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F121 — Resilience & Failure Recovery

WHAT / HOW: Recover from DB, worker, API, AI and sensor failures.

OUTPUT: controlled recovery/degraded state.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F122 — Data Lifecycle

WHAT / HOW: Manage retention, indexing, archive and export.

OUTPUT: lifecycle-controlled data.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F123 — RDRS Self-Security

WHAT / HOW: Least privilege, secret protection, authorization, input/path safety, evidence protection and AI isolation.

OUTPUT: platform security validation.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F124 — Complete Validation & Safe Simulation

WHAT / HOW: Run controlled ransomware-like simulations through telemetry → detection → incident → evidence → response → recovery.

OUTPUT: end-to-end validation evidence.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

F125 — Production Packaging & Submission

WHAT / HOW: Docker, compose, persistent data, logs, evidence, README, architecture, API/CLI docs, screenshots, demo and final ZIP.

OUTPUT: deployable/submission-ready package.

100/100 GATE: implementation works; expected output verified; relevant edge/error cases handled; dependencies integrated; applicable GUI/CLI/API exposed; security/integrity checked; tests pass; documentation updated; then the feature is LOCKED before the next feature begins.

Day Completion & GitHub Pushes

DAY 1 / V1: Features F01–F25 → each individually 100/100 → LOCK → GitHub Push #1.

DAY 2 / V2: Features F26–F50 → each individually 100/100 → LOCK → GitHub Push #2.

DAY 3 / V3: Features F51–F75 → each individually 100/100 → LOCK → GitHub Push #3.

DAY 4 / V4: Features F76–F100 → each individually 100/100 → LOCK → GitHub Push #4.

DAY 5 / V5: Features F101–F125 → each individually 100/100 → LOCK → GitHub Push #5 / FINAL.

Final Acceptance

125/125 individual features accounted for. No feature is intentionally merged away or silently dropped. V1 is not a basic version; it is the first final-quality development milestone. The final product is intended for SOC/company-style adoption rather than a normal internship-only prototype.